Generating Time-Varying and Non-Linear Covariate Effects for Survival Simulation

A piecewise-exponential grid-inversion data-generating process, validated against rpsurv.

Imad El Badisy

2026-06-10

Table of contents

Model specification . . . 2
Piecewise-constant hazard inversion for the time-varying part . . . 2
Shape libraries . . . 3
Implementation . . . 3
Validation with rpsurv . . . 5
    The spline basis . . . 6
    The three link scales . . . 6
    Estimation . . . 7
Uncertainty and absolute contrasts . . . 11
Why the validation is a fair test . . . 12
Cox proportional hazards as a nested special case . . . 13
Effect of spline flexibility (df) on the recovered effect curves . . . 14
Model selection by parsimony . . . 16
Model-fit diagnostics . . . 17
Practical notes . . . 18

Simulating survival data with a non-proportional, time-varying covariate effect (TVE) has no closed-form cumulative hazard in general, so the standard inversion method used for exponential or Weibull data does not apply directly. This note derives a piecewise-exponential (PWE) grid-inversion scheme that sidesteps that problem for an arbitrary time-varying log-hazard ratio, states its exactness and bias properties, extends it to a second, independent source of model complexity, a non-linear (but time-constant) covariate effect (NLE), and validates both against `rpsurv`, a native Royston-Parmar flexible parametric implementation.[1]

[1] `rpsurv` package documentation and source, GitHub: https://github.com/ielbadisy/rpsurv. Royston, P.,

Model specification

For a subject with a binary covariate X_1 carrying a time-varying effect and a continuous covariate X_2 carrying a possibly non-linear effect, the target hazard is

$$h(t \mid x_1, x_2) = \lambda_0 \exp\{\beta_1(t)\, x_1 + g(x_2)\}, \qquad t \in [0, t_{\max}],$$

with constant baseline λ_0, an arbitrary (possibly non-monotone) function $\beta_1(t)$, and an arbitrary function $g(\cdot)$ in place of a linear term $\beta_2 x_2$. The corresponding cumulative hazard, $H(t \mid x_1, x_2) = \int_0^t \lambda_0 \exp\{\beta_1(s)x_1 + g(x_2)\}\, ds$, has no closed form for the $\beta_1(\cdot)$ shapes used below, which motivates a piecewise approximation of the time-varying part only, since $g(x_2)$ does not depend on t and needs no such approximation.

Piecewise-constant hazard inversion for the time-varying part

Partition $[0, t_{\max})$ into $K = t_{\max}/\delta$ half-open intervals $I_i = [t_i, t_i + \delta)$, $t_i = i\delta$, with a small step δ (default 0.01–0.02). Freeze $\beta_1(t) \approx \beta_1(t_i)$ on each interval, so conditional on survival to t_i the hazard is locally constant:

$$h(t \mid x_1, x_2) \approx h_i(x_1, x_2) := \lambda_0 \exp\{\beta_1(t_i)\, x_1 + g(x_2)\}, \quad t \in I_i.$$

Under a locally constant hazard, the residual time from t_i is exactly exponential, so a candidate draw follows by inversion, $\tilde{T}_i = t_i - \log(U_i)/h_i(x_1, x_2)$, $U_i \sim \text{Uniform}(0, 1)$, independently across i and subjects. A candidate is valid only if self-consistent, $t_i \le \tilde{T}_i < t_i + \delta$; the event time is $T_\delta = \min\{\tilde{T}_i : \text{valid}\}$ (a large sentinel if none is valid).

Algorithm 1 (`genTVE_NLE` event-time generation).

1. Draw covariates $X_1 \sim \text{Bernoulli}(0.5)$, $X_2 = |Z|$, $Z \sim N(0, 1)$; compute $g(X_2)$ once, since it is time-constant.
2. For $i = 0, \ldots, K-1$: draw $U_i \sim \text{Uniform}(0, 1)$ and compute $\tilde{T}_i = t_i - \log(U_i)/[\lambda_0 \exp\{\beta_1(t_i)X_1 + g(X_2)\}]$.
3. Retain $\tilde{T}_i$ only if $t_i \le \tilde{T}_i < t_i + \delta$; else discard it.
4. Set $T_\delta = \min_i\{\tilde{T}_i : \text{valid}\}$.

Exactness given the discretized hazard. Write $h_i = h_i(x_1, x_2)$, $p_i = 1 - \exp(-h_i\delta)$, and $A_i = \{\tilde{T}_i \text{ valid}\}$. Since the U_i are i.i.d. and independent across i, the A_i are independent with $P(A_i) = P\{U_i > e^{-h_i\delta}\} = p_i$, exactly the discrete failure probability of a constant hazard h_i over an interval of width δ. Because valid candidates from earlier intervals are always smaller than valid candidates from later ones, taking the minimum over valid candidates is equivalent

and Parmar, M. K. B. (2002). Flexible parametric proportional-hazards and proportional-odds models for censored survival data. *Statistics in Medicine*, 21(15), 2175-2197.

to stopping at the first success in the independent sequence $A_0, A_1, ...$, so T_δ has exactly the distribution implied by the discretized hazard h_δ: the algorithm carries no sampling error beyond the fact that $h_\delta \neq h$.

Discretization bias vanishes as $\delta \to 0$. If $\beta_1(\cdot)$ is Lipschitz with constant L, then $|\beta_1(s) - \beta_1(t_i)| \leq L\delta$ on I_i, so the per-interval hazard error is $O(\delta)$ and the cumulative error over $K = t_{\max}/\delta$ intervals is $|H_\delta(t \mid x) - H(t \mid x)| = O(\delta)$, uniformly on $[0, t_{\max}]$. Uniform convergence of $H_\delta \to H$ gives uniform convergence of $S_\delta = \exp(-H_\delta) \to S$, hence $T_\delta \xrightarrow{d} T$.

Composing a non-linear effect costs nothing extra. The bias bound above is derived entirely from how well $\beta_1(t)x_1$, the *only* time-varying term, is approximated on each interval; $g(x_2)$ enters h_i and h identically, unapproximated, regardless of whether it is linear or not. Replacing $\beta_2 x_2$ with an arbitrary $g(x_2)$ therefore does not change the bound above: the discretization bias remains $O(\delta)$ and depends only on $\beta_1(\cdot)$'s smoothness, never on the functional form of g. This is what makes it possible to add a non-linear covariate effect to Algorithm 1 without deriving anything new, only evaluating a different function inside the same exponent.

Shape libraries

Four $\beta_1(t)$ shapes (reused from the original TVE study) and four analogous $g(x_2)$ shapes, chosen so each pair spans a comparable log-hazard-ratio range and mirrors its counterpart's qualitative behavior:

$\beta_1(t)$ (time)	Behavior	$g(x_2)$ (covariate)	Behavior
`bathub`: $0.10 + 1.20e^{-t/0.5} + 0.04t^2$	high early, dips, rises late	`ushape`: $0.15(x_2 - 1.5)^2$	high at extremes, low in the middle
`decreasing`: $0.32 + 1.42e^{-t} - 0.02t^{0.7}$	monotone decreasing	`saturating`: $0.90\log(1 + x_2)$	diminishing returns
`increasing`: $0.01 + 1.10t^{0.4}$	monotone increasing	`quadratic`: $0.50x_2 - 0.25x_2^2$	rises then falls (inverted U)
`delayed`: $1.30/(1 + e^{-2(t-2.5)})$	near-null, then a logistic emergence	`threshold`: $1.00/(1 + e^{-3(x_2-1.5)})$	near-null, then a logistic step

`bathub` (sic) is the historical argument value in the original implementation and is kept as-is below for continuity with that code.

Implementation

```r
true_beta1 <- function(shape, t) {
  switch(shape,
    bathub     = 0.10 + 1.20 * exp(-t / 0.5) + 0.04 * t^2,
    decreasing = 0.32 + 1.42 * exp(-t) - 0.02 * t^0.7,
    increasing = 0.01 + 1.10 * t^0.4,
    delayed    = 1.30 / (1 + exp(-2 * (t - 2.5))),
    constant   = rep(0.70, length(t))   # proportional hazards
  )
}
true_g <- function(shape, x2) {
  switch(shape,
    quadratic  = 0.50 * x2 - 0.25 * x2^2,
    saturating = 0.90 * log1p(x2),
    ushape     = 0.15 * (x2 - 1.5)^2,
    threshold  = 1.00 / (1 + exp(-3 * (x2 - 1.5))),
    linear     = 0.30 * x2              # standard log-linear effect
  )
}

#' Generate survival data with a time-varying effect and a non-linear covariate effect
#' @param n number of subjects
#' @param maxt administrative censoring horizon
#' @param tve_shape one of "bathub", "decreasing", "increasing", "delayed", "constant"
#' @param nle_shape one of "quadratic", "saturating", "ushape", "threshold", "linear"
#' @param cens random-censoring setting, labelled by its nominal rate: 0.1, 0.3, or 0.5
#'   (the realized rate also includes administrative censoring at maxt)
#' @param delta grid step for the piecewise-constant hazard approximation
genTVE_NLE <- function(n = 1000, maxt = 5, tve_shape = "bathub", nle_shape = "quadratic",
                       cens = 0.3, delta = 0.02) {
  x1  <- rbinom(n, 1, 0.5)      # binary, TVE covariate
  x2  <- abs(rnorm(n))          # continuous, NLE covariate
  gx2 <- true_g(nle_shape, x2)  # time-constant, computed once

  lambda   <- 0.3
  lambda_c <- switch(as.character(cens), "0.1" = 0.00001, "0.3" = 0.2, "0.5" = 0.5)
  time_grid <- seq(0, maxt - delta, delta)

  # one candidate draw per subject per interval (Algorithm 1, steps 2-3):
  # valid only if it falls back inside the interval that generated it
  candidates <- matrix(NA_real_, nrow = n, ncol = length(time_grid))
  for (i in seq_along(time_grid)) {
    t_i <- time_grid[i]
```

```
    hazard_i <- lambda * exp(true_beta1(tve_shape, t_i) * x1 + gx2)
    draw <- t_i - log(runif(n)) / hazard_i
    valid <- draw >= t_i & draw < t_i + delta
    candidates[valid, i] <- draw[valid]
  }

  # Algorithm 1, step 4; Inf when no valid draw falls before maxt
  event_time <- apply(candidates, 1, function(z) if (all(is.na(z))) Inf else min(z, na.rm = T

  censor_time <- -log(runif(n)) / lambda_c
  status <- as.integer(event_time <= pmin(censor_time, maxt))
  time   <- pmin(event_time, censor_time, maxt)

  data.frame(id = seq_len(n), time = time, status = status, x1 = x1, x2 = x2)
}
```

Setting `nle_shape = "linear"` recovers the original TVE-only generator, with $g(x_2) = 0.3\,x_2$; setting `tve_shape = "constant"` as well gives a proportional-hazards model with constant $\beta_1 = 0.7$. Both extensions are strict generalizations of the original single-mechanism DGP, not separate code paths. A subject with no valid draw before `maxt` has $T_\delta = \infty$ and is administratively censored at `maxt`; the event indicator is therefore $\mathbb{1}\{T_\delta \le \min(C, t_{\max})\}$, not $\mathbb{1}\{T_\delta < C\}$, so that a long random-censoring time C never turns an administratively censored subject into an event.

Validation with rpsurv

`rpsurv::rpsurv()` fits the Royston-Parmar family: the log cumulative hazard is $\log H(t \mid x_1, x_2) = s_0(\log t) + x_1 s_1(\log t) + \eta(x_2)$, where s_0 is a restricted cubic spline in $\log t$ (argument `df`), s_1 is a second spline in $\log t$ requested via `tve = "x1"` (argument `tve.df`), and $\eta(x_2)$ is a non-linear covariate term represented by an ordinary spline basis in x_2 (`splines::ns()`) placed directly in the model formula. `rpsurv`'s `tve` argument name is deliberate: it flags a **time-varying effect**, not a time-varying **covariate** (a value that itself changes over follow-up, represented instead through counting-process data, unrelated to what is validated here). The remainder of this section spells out exactly what `rpsurv()` fits and how, so that the mapping from the DGP above to the estimator validating it has no unstated step.

The spline basis

s_0 and s_1 (any `tve` term) share the same construction internally, a restricted (natural) cubic spline in the Durrleman-Simon parameterization used by Royston and Parmar (2002).[2] The NLE term $\eta(x_2)$ in this note is a separate mechanism: it is `splines::ns()`, R's own natural-cubic-spline basis (B-spline parameterization, not the Durrleman-Simon one below), computed outside `rpsurv()` and passed in as ordinary covariate columns, so `rpsurv()` itself never distinguishes it from an ordinary linear term; the two constructions are functionally similar (both restricted/natural cubic splines) but not the same code path. Given ordered knots $k_0 < k_1 < \cdots < k_m < k_{m+1}$ (k_0, k_{m+1} the boundary knots at the min/max of the argument; $k_1, \ldots, k_m$ the m interior knots, $m = \texttt{df} - 1$), the basis is a linear term plus one non-linear term per interior knot,

$$s(u) = \gamma_0 + \gamma_1 u + \sum_{j=1}^{m} \gamma_{j+1}\, v_j(u), \qquad v_j(u) = (u - k_j)_+^3 - \lambda_j (u - k_0)_+^3 - (1 - \lambda_j)(u - k_{m+1})_+^3,$$

with $(a)_+ = \max(a, 0)$ and $\lambda_j = (k_{m+1} - k_j)/(k_{m+1} - k_0)$. Each v_j is an ordinary cubic on $[k_0, k_{m+1}]$, and the specific combination of the three truncated-cubic terms is chosen so that v_j, and hence s, is exactly linear outside $[k_0, k_{m+1}]$ (both the cubic and quadratic terms of v_j cancel there), which is what keeps the fitted hazard from doing anything erratic beyond the observed follow-up range, the defining feature of a *restricted* (as opposed to an unconstrained) cubic spline. The basis has $m + 1 = \texttt{df}$ non-intercept columns; together with a separate intercept term, a spline of degrees of freedom `df` therefore contributes `df` $+ 1$ free parameters. For s_0/s_1, both the boundary knots (min and max) and the interior knots (equally spaced quantiles) default to being computed from log *event* times only, excluding censored observations entirely from the knot-placement step, matching the convention used by `rstpm2::stpm2()` and `flexsurv::flexsurvspline()`; an interior knot that lands exactly on a boundary knot (event times piled up near one end of follow-up) makes that basis column identically zero, so `rpsurv` drops it and warns, reducing the realized spline `df` below the requested value rather than fitting a silently rank-deficient model.

The three link scales

`scale` chooses how $\eta(t, x) = s_0(\log t) + x_1 s_1(\log t) + \eta(x_2)$ (the full linear predictor, spline terms plus any ordinary covariate terms) maps to survival:

[2] `rpsurv` package documentation and source, GitHub: https://github.com/ielbadisy/rpsurv. Royston, P., and Parmar, M. K. B. (2002). Flexible parametric proportional-hazards and proportional-odds models for censored survival data. *Statistics in Medicine*, 21(15), 2175-2197.

$$\texttt{"hazard"}\ \text{(proportional hazards)}: \quad S(t \mid x) = \exp\{-\exp(\eta)\}, \quad H(t \mid x) = \exp(\eta)$$
$$\texttt{"odds"}\ \text{(proportional odds)}: \quad S(t \mid x) = \frac{1}{1+\exp(\eta)}, \qquad \frac{1-S}{S} = \exp(\eta)$$
$$\texttt{"normal"}\ \text{(probit)}: \quad S(t \mid x) = \Phi(-\eta).$$

`"hazard"` is what every use of `rpsurv()` in this note relies on (it is the model whose nesting of the Cox model and whose hazard-ratio contrast are used throughout); `"odds"` and `"normal"` are alternative link functions for the same spline-on-linear-predictor idea, not used here, included for completeness since `scale` is a real, user-facing argument. The hazard itself follows by differentiating $H(t \mid x) = -\log S(t \mid x)$ with respect to t via the chain rule through $\log t$: with $\eta'(\log t)$ denoting $d\eta/d(\log t)$ (evaluated by differentiating the spline basis directly, `derivative = TRUE` in the basis constructor, not by numerical differentiation),

$$h(t \mid x) = \frac{dH}{dt} = \frac{dH}{d\eta} \cdot \frac{d\eta}{d\log t} \cdot \frac{d\log t}{dt} = \frac{dH}{d\eta} \cdot \eta'(\log t) \cdot \frac{1}{t},$$

which for the hazard scale ($dH/d\eta = \exp(\eta)$) gives $h = \eta'(\log t)/t \cdot \exp(\eta)$. Because s_1 acts on the log *cumulative* hazard, it is not itself the log hazard ratio: for $x_1 = 1$ versus $x_1 = 0$ at the same x_2,

$$\beta_1(t) = \log \mathrm{HR}(t) = s_1(\log t) + \log\left\{1 + \frac{s_1'(\log t)}{s_0'(\log t)}\right\},$$

which reduces to $s_1(\log t)$ only when s_1 is constant (proportional hazards). Every estimate of $\beta_1(t)$ below is therefore taken as $\log \widehat{\mathrm{HR}}(t)$ from `predict(type = "hr")`, which evaluates this ratio of hazards, never as $\hat{s}_1(\log t)$.

Estimation

Parameters (every spline coefficient across s_0, s_1, and any covariate spline, plus ordinary linear-term coefficients) are collected into one vector β and fit by direct maximization of the exact parametric log-likelihood, not a partial likelihood: for right-censored data without truncation, writing $\eta_i = \eta(t_i, x_i; \beta)$ at each subject's own observed time,

$$\ell(\beta) = \sum_{i=1}^{n} \Big[\delta_i \log h(t_i \mid x_i) + \log S(t_i \mid x_i)\Big],$$

the standard survival log-likelihood: an event contributes its log-hazard plus log-survival (density $= h \cdot S$ in log form), a censored observation contributes only log-survival. For left-truncated (counting-process) data, each row's contribution is conditioned on survival to its own entry time

t_i^{entry}, replacing $\log S(t_i)$ with $\log S(t_i) - \log S(t_i^{\text{entry}})$ in the expression above, which is exactly the mechanism that lets `rpsurv` fit genuine time-varying covariates through `Surv(start, stop, status)` data without any special-casing beyond this per-row conditioning term. $\ell(\beta)$ is maximized by `stats::optim(method = "BFGS")` using an analytic gradient (implemented in C++, parallelized across subjects with `RcppParallel::parallelReduce`), not a derivative-free or expectation-maximization scheme; `logLik()`, `AIC()`, and `BIC()` used throughout this note are read directly off this $\ell(\hat{\beta})$ and the count of free parameters in β, with no further approximation.

`predict.rpsurv(..., type = "hr")` returns the model-implied instantaneous hazard ratio between a contrast profile (`newdata`) and a reference profile (`newdata0`) directly, computed internally as the ratio of two `type = "hazard"` predictions, the correct contrast under a time-varying effect (a naive `exp(eta1 - eta0)` is not the instantaneous hazard ratio once `tve` makes the derivative term $s_1'(\log t)$ nonzero). `type = "sdiff"` returns the survival difference the same way, and `rmst_diff()` integrates it to a restricted-mean-survival-time contrast; both support `se.fit = TRUE` via closed-form delta-method gradients. A separate practical point: because `predict.rpsurv()` requires `newdata` to contain the model's covariate columns exactly as stored at fit time, and a spline term is stored as its expanded basis columns, predicting $g(x_2)$ at new x_2 values requires re-evaluating the *same* `ns()` basis object used at fit time (via `predict()` on the basis, not by refitting `ns()` on the new grid) and supplying those columns by name.

```
library(rpsurv)
library(survival)
library(splines)
library(ggplot2)
library(patchwork)
```

```
# R = 100 replications, n = 1500, tve = bathub, nle = quadratic.
# tve.df/df = 6 (matching the flexibility used to validate the original,
# NLE-free version of this DGP) rather than 3/4: at low tve.df the sharp
# late rise of the bathub shape is under-resolved (flattens out past t=3).
R <- 100
n <- 1500
tve_shape <- "bathub"
nle_shape <- "quadratic"

t_grid  <- seq(0.3, 4.6, length.out = 40)
x2_grid <- seq(0, 3, length.out = 20)

est_beta1 <- matrix(NA_real_, R, length(t_grid))
est_g     <- matrix(NA_real_, R, length(x2_grid))
```

```r
set.seed(2026)
for (r in seq_len(R)) {
  dat <- genTVE_NLE(n = n, tve_shape = tve_shape, nle_shape = nle_shape)

  basis <- ns(dat$x2, df = 3)
  B <- predict(basis, dat$x2); colnames(B) <- paste0("x2_ns", 1:3)
  fitdat <- data.frame(time = dat$time, status = dat$status, x1 = dat$x1, B, check.names = FA

  fit <- tryCatch(
    rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
           data = fitdat, df = 6, tve = "x1", tve.df = 6),
    error = function(e) NULL
  )
  if (is.null(fit)) next

  mkbasis <- function(x2v) {
    b <- predict(basis, x2v); colnames(b) <- paste0("x2_ns", 1:3); b
  }

  nd1 <- data.frame(x1 = 1, mkbasis(0), check.names = FALSE)
  nd0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)
  hr  <- predict(fit, newdata = nd1, newdata0 = nd0, times = t_grid, type = "hr")$est
  est_beta1[r, ] <- log(hr)

  nd_g  <- data.frame(x1 = 0, mkbasis(x2_grid), check.names = FALSE)
  nd_g0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)
  link_grid <- predict(fit, newdata = nd_g,  times = 1, type = "link")$est
  link_0    <- predict(fit, newdata = nd_g0, times = 1, type = "link")$est
  est_g[r, ] <- link_grid - link_0
}
```

```r
truth_beta1 <- true_beta1(tve_shape, t_grid)
truth_g     <- true_g(nle_shape, x2_grid) - true_g(nle_shape, 0)
mean_b <- colMeans(est_beta1, na.rm = TRUE)
mean_g <- colMeans(est_g, na.rm = TRUE)

data.frame(
  target  = c("TVE: beta1(t), bathub", "NLE: g(x2), quadratic"),
  n_valid = c(sum(rowSums(is.na(est_beta1)) == 0), sum(rowSums(is.na(est_g)) == 0)),
  mean_abs_error = round(c(mean(abs(mean_b - truth_beta1)), mean(abs(mean_g - truth_g))), 3)
)
```

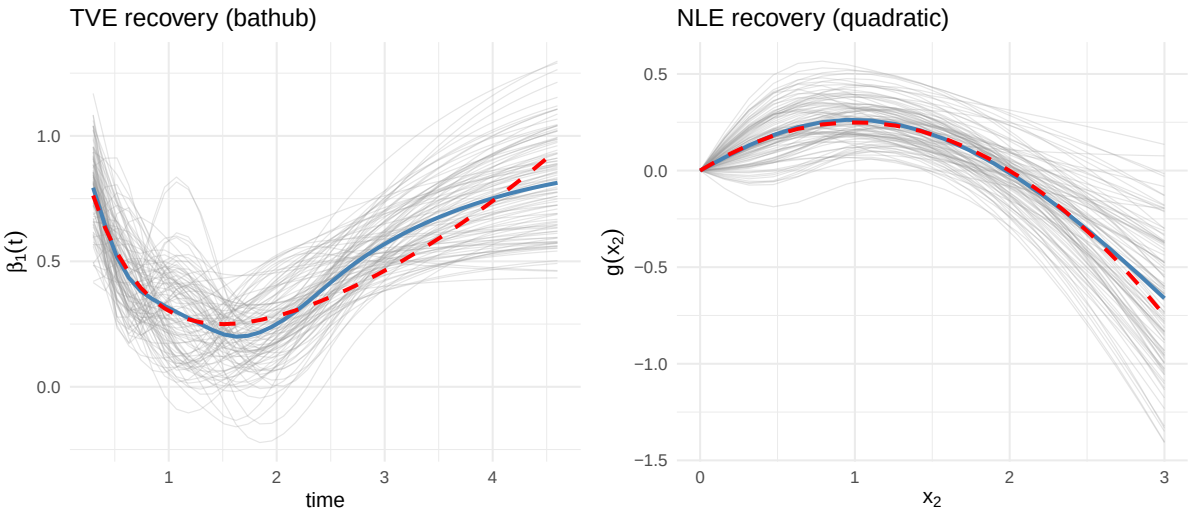
```
                  target n_valid mean_abs_error
1 TVE: beta1(t), bathub     100          0.049
2 NLE: g(x2), quadratic     100          0.015
```

Each panel below shows all $R = 100$ individual replicate curves (thin, semi-transparent), their mean (solid), and the true generating curve (dashed), the same convention used for the validation figures in the original TVE-only study.

```
long_curves <- function(mat, grid, xname) {
  data.frame(rep = rep(seq_len(nrow(mat)), each = ncol(mat)),
             x   = rep(grid, nrow(mat)),
             y   = as.vector(t(mat))) |> setNames(c("rep", xname, "y"))
}

panel <- function(mat, grid, truth, xname, xlab, ylab, title) {
  ggplot(long_curves(mat, grid, xname), aes(.data[[xname]], y, group = rep)) +
    geom_line(color = "grey60", alpha = 0.25, linewidth = 0.3) +
    geom_line(data = data.frame(x = grid, y = colMeans(mat, na.rm = TRUE)),
              aes(x, y, group = NULL), color = "steelblue", linewidth = 1) +
    geom_line(data = data.frame(x = grid, y = truth),
              aes(x, y, group = NULL), color = "red", linewidth = 1, linetype = "dashed") +
    labs(x = xlab, y = ylab, title = title) +
    theme_minimal(base_size = 11)
}

p1 <- panel(est_beta1, t_grid, truth_beta1, "t",  "time", expression(beta[1](t)), "TVE recove
p2 <- panel(est_g, x2_grid, truth_g, "x2", expression(x[2]), expression(g(x[2])), "NLE recove
p1 + p2
```



The NLE recovery is close to exact throughout, consistent with $g(x_2)$ entering the likelihood the same way at every event time, an ordinary spline-regression problem with no time-discretization step involved. The individual TVE curves show the estimator's true finite-sample variability directly rather than hiding it behind a band: most replicates track the bathtub shape's dip and late rise, some flatten out past $t = 3$ where events are sparser, and the mean over all 100 tracks the true curve closely through most of follow-up, with a modest, expected residual gap right at the latest evaluation times, the same pattern documented for the original, NLE-free version of this DGP, attributable to estimation noise in a spline fit at the boundary of the observed data rather than to a mis-specified generating hazard, since the discretization-bias argument above places no such time-dependence on the DGP's own error.

Uncertainty and absolute contrasts

The point estimates above are one part of the picture; `rpsurv` also returns delta-method confidence intervals for the hazard ratio and, on the absolute scale, the survival difference and its integral, the restricted mean survival time (RMST) difference, refit here on one replicate for illustration.

```
set.seed(11)
dat <- genTVE_NLE(n = n, tve_shape = tve_shape, nle_shape = nle_shape)
basis <- ns(dat$x2, df = 3)
B <- predict(basis, dat$x2); colnames(B) <- paste0("x2_ns", 1:3)
fitdat <- data.frame(time = dat$time, status = dat$status, x1 = dat$x1, B, check.names = FALS
fit <- rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
              data = fitdat, df = 6, tve = "x1", tve.df = 6)

mkbasis <- function(x2v) { b <- predict(basis, x2v); colnames(b) <- paste0("x2_ns", 1:3); b
nd1 <- data.frame(x1 = 1, mkbasis(0), check.names = FALSE)
nd0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)

# hazard ratio with a 95% delta-method CI
hr <- predict(fit, newdata = nd1, newdata0 = nd0, times = c(1, 2, 3, 4), type = "hr", se.fit
round(hr, 3)
```

```
  id time   est lower upper
1  1    1 1.500 1.067 2.108
2  1    2 1.385 0.946 2.028
3  1    3 1.477 1.019 2.140
4  1    4 1.515 0.920 2.494
```

```r
# restricted mean survival time difference up to tau = 4, with SE and CI
rmst_diff(fit, newdata = nd1, newdata0 = nd0, tau = 4, se.fit = TRUE)
```

```
        est        se      lower      upper
1 -0.679012 0.1155692 -0.9055235 -0.4525005
```

Under the `bathub` shape the true hazard ratios at $t = 1, 2, 3, 4$ are $1.35, 1.33, 1.59, 2.09$. In this replicate the point estimates (about 1.4-1.5) are on the risk-increasing side at every landmark, but a single sample of $n = 1500$ is not enough to resolve the contrast at every time: the 95% CI excludes 1 at $t = 1$ and $t = 3$ and includes it at $t = 2$, near the bathtub minimum, and at $t = 4$, where late follow-up is sparse and the interval is widest. The late underestimate (1.52 against a true 2.09) is the same boundary flattening seen in the recovery plot above. The RMST difference up to $\tau = 4$ is negative with a CI that excludes 0: $x_1 = 1$ patients accumulate less restricted mean survival time than $x_1 = 0$ patients over that window. Integrated over follow-up, the effect is clearly detectable even where the pointwise hazard ratio is not, which is the usual argument for reporting an absolute, cumulative contrast alongside a time-varying hazard ratio. `rmst_diff()`'s standard error is not a bootstrap: it applies the delta method directly to the trapezoidal-weighted sum of the survival-difference gradient, exploiting that a linear combination of the (asymptotically normal) model coefficients is itself normal, so a single quadratic form gives the SE without resampling.

Why the validation is a fair test

Both `rpsurv`'s spline representation and Algorithm 1's histogram representation are sieve approximations to the same non-proportional-hazards surface, at different resolutions: `rpsurv` uses a natural-spline basis in $\log t$ (resolution `df`/`tve.df`), Algorithm 1 uses a piecewise-constant basis in raw t (resolution δ). As the spline's knot count grows with n, the span of natural cubic splines becomes dense in the space of smooth functions on the observed time range, so $\hat{\beta}_1(t) \xrightarrow{p} \beta_1(t)$ pointwise under standard sieve-consistency arguments, provided $\beta_1(t)$ is continuous, true for all four shapes here. The same logic, applied to an ordinary regression spline in a covariate rather than in log time, is exactly why $\hat{g}(x_2) \xrightarrow{p} g(x_2)$: there is nothing special about a spline being indexed by time rather than by a covariate value for this argument to hold. The standard proportional-hazards Cox model is the nested special case $s_1 \equiv 0$ and $\eta(x_2) = \beta_2 x_2$, so a demonstration that the flexible model recovers non-constant $\beta_1(t)$ and non-linear $g(x_2)$ already implies, a fortiori, that it recovers their constant/linear special cases.

Cox proportional hazards as a nested special case

The previous section states the nesting argument abstractly; here it is checked directly. Fit the same proportional-hazards data with `coxph()`[3] and with `rpsurv()` under `scale = "hazard"` and no `tve` term, so $s_1 \equiv 0$ and the model reduces to $\log H(t \mid x) = s_0(\log t) + \beta^\top x$, a spline-baseline analogue of the Cox model rather than a distinct model family.

```
set.seed(1)
n_ph <- 1000
x1 <- rbinom(n_ph, 1, 0.5)
x2 <- abs(rnorm(n_ph))
beta1_true <- 0.7; beta2_true <- 0.3
t_ph <- rexp(n_ph, rate = 0.3 * exp(beta1_true * x1 + beta2_true * x2))
status_ph <- as.integer(t_ph <= 5)
t_ph <- pmin(t_ph, 5)
dph <- data.frame(time = t_ph, status = status_ph, x1 = x1, x2 = x2)

fit_cox <- coxph(Surv(time, status) ~ x1 + x2, data = dph)
fit_rp  <- rpsurv(Surv(time, status) ~ x1 + x2, data = dph, df = 4)

data.frame(
  term        = c("x1", "x2"),
  true        = c(beta1_true, beta2_true),
  coxph_coef  = round(coef(fit_cox), 4),
  coxph_se    = round(sqrt(diag(vcov(fit_cox))), 4),
  rpsurv_coef = round(coef(fit_rp)[c("x1", "x2")], 4),
  rpsurv_se   = round(sqrt(diag(vcov(fit_rp)))[c("x1", "x2")], 4)
)
```

```
   term true coxph_coef coxph_se rpsurv_coef rpsurv_se
x1   x1  0.7     0.6770   0.0682      0.6780    0.0682
x2   x2  0.3     0.2379   0.0519      0.2394    0.0519
```

`coxph()` estimates β by partial likelihood, profiling the baseline hazard out entirely; `rpsurv` instead estimates β jointly with an explicit `df = 4` spline for s_0. The two are different estimators of the same target, not the same computation, so exact numerical agreement is not guaranteed, only asymptotic agreement under a correctly specified baseline. The point estimates agree to within 0.002 and the standard errors to four decimal places, which is what the nesting argument predicts: with the baseline hazard genuinely constant (an exponential DGP here) and `df = 4`

[3] Cox, D. R. (1972). Regression models and life tables (with discussion). *Journal of the Royal Statistical Society, Series B*, 34(2), 187-220.

more than flexible enough to represent a constant, `rpsurv`'s spline-baseline model recovers the same proportional-hazards fit as Cox regression.

Effect of spline flexibility (df) on the recovered effect curves

`tve.df` and the degrees of freedom of the covariate spline are resolution knobs for two *different* curves, $\beta_1(t)$ and $g(x_2)$, not for $S(t)$: a survival curve can look fine even when the effect curve underneath it is badly resolved, since $S(t)$ pools information over the whole follow-up window while $\beta_1(t)$ and $g(x_2)$ are local functions that a coarse spline can flatten out or over-wiggle without moving $S(t)$ very much. The demonstration therefore targets the effect curves directly, on one fixed TVE+NLE dataset (`bathub` and `quadratic`, as validated above).

```
set.seed(7)
dat_df <- genTVE_NLE(n = 1500, tve_shape = "bathub", nle_shape = "quadratic")
basis3 <- ns(dat_df$x2, df = 3)
B3 <- predict(basis3, dat_df$x2); colnames(B3) <- paste0("x2_ns", 1:3)
fitdat3 <- data.frame(time = dat_df$time, status = dat_df$status, x1 = dat_df$x1, B3, check.r
mkbasis3 <- function(x2v) { b <- predict(basis3, x2v); colnames(b) <- paste0("x2_ns", 1:3); b
```

Holding the NLE spline at `df = 3` and the baseline at `df = 6`, `tve.df` is varied over a grid and the recovered $\widehat{\beta}_1(t) = \log \widehat{HR}(t)$ (contrasting $x_1 = 1$ against $x_1 = 0$ at $x_2 = 0$) is compared to the known truth.

```
t_grid_df <- seq(0.3, 4.6, length.out = 60)
truth_beta1_df <- true_beta1("bathub", t_grid_df)

tve_df_grid <- c(2, 3, 6, 10)
beta1_curves <- lapply(tve_df_grid, function(td) {
  fit <- rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
                data = fitdat3, df = 6, tve = "x1", tve.df = td)
  nd1 <- data.frame(x1 = 1, mkbasis3(0), check.names = FALSE)
  nd0 <- data.frame(x1 = 0, mkbasis3(0), check.names = FALSE)
  log(predict(fit, newdata = nd1, newdata0 = nd0, times = t_grid_df, type = "hr")$est)
})
mae_tve_df <- sapply(beta1_curves, function(e) mean(abs(e - truth_beta1_df)))
data.frame(tve.df = tve_df_grid, mean_abs_error = round(mae_tve_df, 4))
```

```
  tve.df mean_abs_error
1      2         0.1855
2      3         0.1331
3      6         0.0780
4     10         0.1002
```

Separately, holding `tve.df = 6` fixed, the NLE covariate spline's own `df` is varied and the recovered $\hat{g}(x_2)$ (centered at $x_2 = 0$, since only the shape, not the arbitrary intercept, is identified) is compared to the known `quadratic` truth.

```
x2_grid_df <- seq(0, 3, length.out = 30)
truth_g_df <- true_g("quadratic", x2_grid_df) - true_g("quadratic", 0)

nle_df_grid <- c(2, 3, 5, 8)
g_curves <- lapply(nle_df_grid, function(gd) {
  basis_k <- ns(dat_df$x2, df = gd)
  Bk <- predict(basis_k, dat_df$x2); colnames(Bk) <- paste0("x2_ns", 1:gd)
  fitdatk <- data.frame(time = dat_df$time, status = dat_df$status, x1 = dat_df$x1, Bk, check
  fml <- as.formula(paste("Surv(time, status) ~ x1 +", paste0("x2_ns", 1:gd, collapse = " + "
  fit <- rpsurv(fml, data = fitdatk, df = 6, tve = "x1", tve.df = 6)
  mkbasisk <- function(x2v) { b <- predict(basis_k, x2v); colnames(b) <- paste0("x2_ns", 1:gd
  nd_g  <- data.frame(x1 = 0, mkbasisk(x2_grid_df), check.names = FALSE)
  nd_g0 <- data.frame(x1 = 0, mkbasisk(0), check.names = FALSE)
  predict(fit, newdata = nd_g, times = 1, type = "link")$est -
    predict(fit, newdata = nd_g0, times = 1, type = "link")$est
})
mae_nle_df <- sapply(g_curves, function(e) mean(abs(e - truth_g_df)))
data.frame(nle.df = nle_df_grid, mean_abs_error = round(mae_nle_df, 4))
```

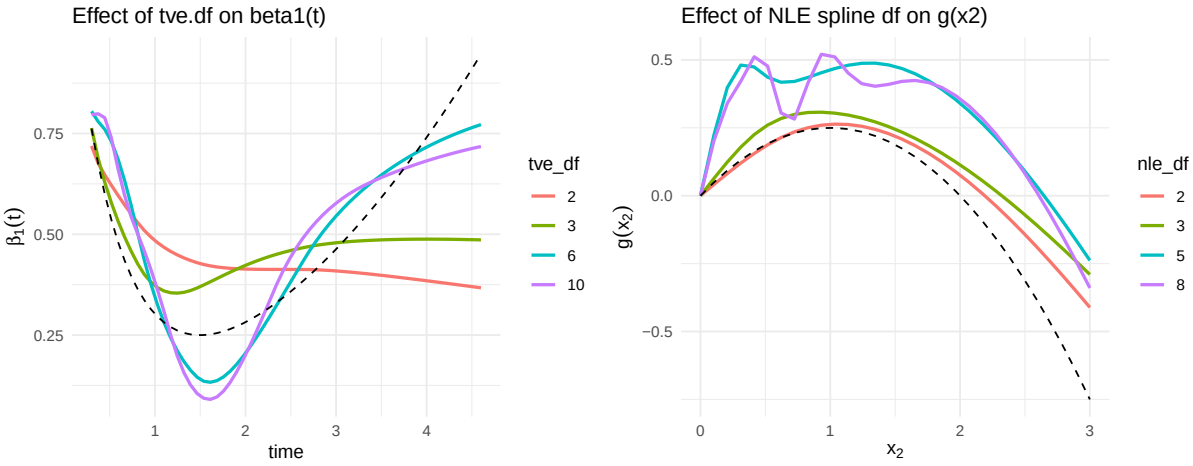
```
  nle.df mean_abs_error
1      2         0.0782
2      3         0.1283
3      5         0.3059
4      8         0.2797
```

```
long_beta1_df <- do.call(rbind, lapply(seq_along(tve_df_grid), function(i)
  data.frame(t = t_grid_df, beta1 = beta1_curves[[i]], tve_df = factor(tve_df_grid[i]))))
p1 <- ggplot(long_beta1_df, aes(t, beta1, color = tve_df)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = data.frame(t = t_grid_df, beta1 = truth_beta1_df), aes(t, beta1),
            color = "black", linetype = "dashed", inherit.aes = FALSE) +
  labs(title = "Effect of tve.df on beta1(t)", x = "time", y = expression(beta[1](t))) +
  theme_minimal(base_size = 11)

long_g_df <- do.call(rbind, lapply(seq_along(nle_df_grid), function(i)
  data.frame(x2 = x2_grid_df, g = g_curves[[i]], nle_df = factor(nle_df_grid[i]))))
p2 <- ggplot(long_g_df, aes(x2, g, color = nle_df)) +
  geom_line(linewidth = 0.9) +
```

```
  geom_line(data = data.frame(x2 = x2_grid_df, g = truth_g_df), aes(x2, g),
            color = "black", linetype = "dashed", inherit.aes = FALSE) +
  labs(title = "Effect of NLE spline df on g(x2)", x = expression(x[2]), y = expression(g(x[2
  theme_minimal(base_size = 11)

p1 + p2
```



For $\beta_1(t)$, `tve.df = 2` and `3` (red, green) flatten through the middle of follow-up and miss the late rise entirely; `tve.df = 6` (cyan) has the lowest error and the right shape, although on this single dataset it overshoots the depth of the dip and still falls short of the late rise; `tve.df = 10` (purple) follows nearly the same curve with a higher error, so the extra knots add variance without adding fidelity. For $g(x_2)$ the true shape is a simple downward-opening parabola, and `nle.df = 2` (red) gives the lowest error; it reproduces the shape over the bulk of the data but sits above the truth for $x_2 > 1.5$, where $X_2 = |Z|$ is sparse. Higher `nle.df` (`3`, `5`, `8`) does not reduce that gap; `5` and `8` add spurious wiggles at low x_2 and roughly double or triple the error, with `5` slightly worse than `8` on this dataset. The practical lesson differs for the two curve types: under-resolving a genuinely curved effect biases it toward flat, while over-resolving a simple effect mostly inflates its variance, so the right `df` depends on the true complexity of the specific effect, not on one rule of thumb applied to both. These are single-dataset illustrations; the replicated validation above is the evidence on average behavior.

Model selection by parsimony

`AIC.rpsurv()` and `BIC.rpsurv()` formalize the `tve.df` choice directly from the fitted log-likelihood, penalizing extra spline coefficients rather than requiring the known $\beta_1(t)$ used above.[4]

[4] Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723. Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*,

```r
parsimony_tab <- do.call(rbind, lapply(tve_df_grid, function(td) {
  fit <- rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
                data = fitdat3, df = 6, tve = "x1", tve.df = td)
  data.frame(tve.df = td, loglik = round(as.numeric(logLik(fit)), 2),
             AIC = round(AIC(fit), 2), BIC = round(BIC(fit), 2))
}))
parsimony_tab
```

```
  tve.df   loglik     AIC     BIC
1      2 -1682.84 3391.68 3455.31
2      3 -1682.01 3392.02 3460.55
3      6 -1679.43 3392.86 3476.07
4     10 -1679.29 3400.58 3503.37
```

This is worth reading carefully against the curve-recovery result above, because the two do not agree here. Curve recovery (MAE) preferred `tve.df = 6`; AIC and BIC both prefer the smallest candidate, `tve.df = 2`, and both increase with `tve.df` from there. The reason is that the extra spline coefficients between `tve.df = 2` and `tve.df = 6` improve the log-likelihood only marginally, by about 3.4 nats on 1500 subjects, well within the noise a `2 x tve.df` AIC penalty (and the stricter `log(n_events) x tve.df` BIC penalty) is designed to discount. AIC and BIC answer "does this extra flexibility earn its keep in the likelihood of the observed event times," which is a real and useful question, but it is not the same question as "does this flexibility recover the true shape of $\beta_1(t)$," and a time-varying effect can be genuine, and even visible in a curve-recovery check against a known truth, while still being too weak a signal in a finite, moderately censored sample for the likelihood to reward modeling it. In practice this means AIC/BIC are a necessary check against overfitting, not a substitute for subject-matter judgment about whether a time-varying or non-linear effect is scientifically expected in the first place; a purely likelihood-driven `tve.df` selection here would have under-modeled a real, moderate-sized effect.

Model-fit diagnostics

Once a working `tve.df` is chosen (here, `tve.df = 6`, on the strength of the curve-recovery check above, since the AIC/BIC comparison alone would have under-selected it), `rpsurv` provides two residual-based checks that do not require knowing the true generating hazard, unlike the simulation-based validation earlier in this note. `residuals(fit, type = "coxsnell")` returns $r_i = \hat{H}(t_i \mid x_i)$;[5] under a correctly specified model these behave like a censored sample

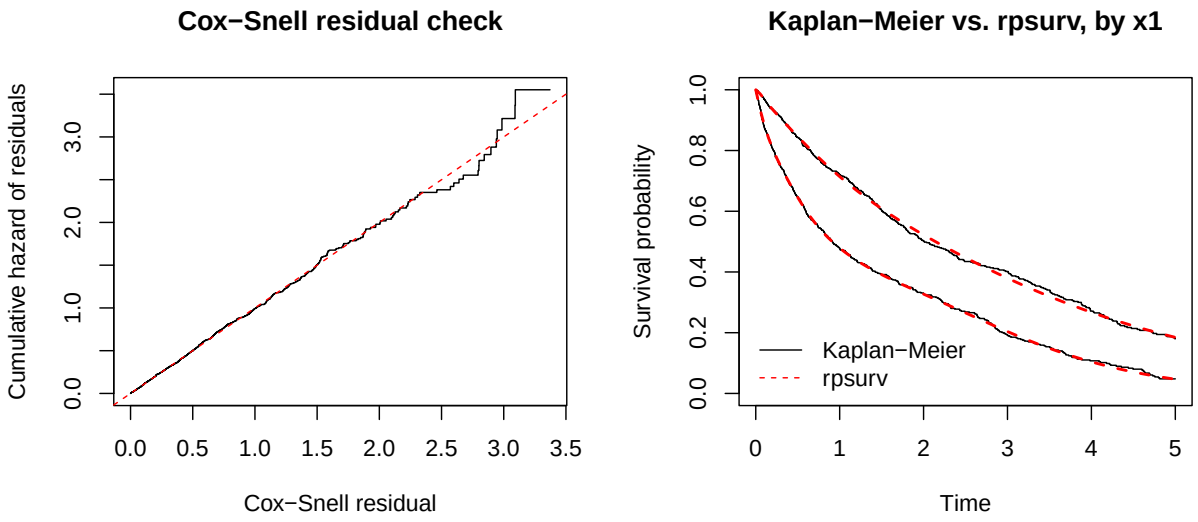
6(2), 461-464.

[5] Cox, D. R., and Snell, E. J. (1968). A general definition of residuals. *Journal of the Royal Statistical Society, Series B*, 30(2), 248-275.

from a unit-exponential distribution, so their own Nelson-Aalen cumulative hazard plotted against themselves should track the 45-degree line. `km_compare_plot()` instead overlays the nonparametric Kaplan-Meier curve against the model's fitted survival curve directly on the outcome scale, here stratified by x_1 to check the fit within each arm separately rather than only on average.

```
fit_diag <- rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
                   data = fitdat3, df = 6, tve = "x1", tve.df = 6)

par(mfrow = c(1, 2))
coxsnell_plot(fit_diag, main = "Cox-Snell residual check")
km_compare_plot(fit_diag, by = "x1", main = "Kaplan-Meier vs. rpsurv, by x1")
```



Both diagnostics support the chosen model: the Cox-Snell points track the reference line closely across the observed residual range with no systematic curvature, and the fitted survival curve overlays the Kaplan-Meier step function within both the $x_1 = 0$ and $x_1 = 1$ strata throughout follow-up, including through the crossing/compression pattern induced by the time-varying effect. Neither diagnostic proves the effect curves were recovered exactly right on their own terms, an overly smooth or overly wiggly $\beta_1(t)$ can both leave these plots looking reasonable at this sample size, which is exactly why the previous two sections used a known DGP for a direct check rather than relying on residual diagnostics alone; but a poor Cox-Snell or KM-vs-model fit would have been a reliable warning sign, and both remain available on real data where, unlike here, the ground truth is not.

Practical notes

Composing a time-varying effect with a non-linear covariate effect requires no change to Algorithm 1's exactness or bias arguments, only evaluating a different, still time-constant,

function inside the same per-interval hazard; this is the main structural point of this note, not an incidental convenience. `rpsurv`'s `tve` argument names the mechanism it targets directly, a time-varying **effect**, not a time-varying **covariate** (a distinct mechanism, represented in `rpsurv` through counting-process `Surv(start, stop, status)` data instead). The one implementation detail worth carrying into any downstream simulation code is that `predict.rpsurv()` expects spline-expanded covariate columns in `newdata` by name, not the raw covariate, so any prediction grid for a spline term needs to reuse the fitted basis object rather than reconstructing it from scratch; this is a real, still-open rough edge, tracked as a follow-up. `type = "hr"`, `type = "sdiff"`, `rmst_diff()`, and delta-method `se.fit` for all three were added to `rpsurv` (0.7.0) while writing this note, once it became clear the manual ratio-of-hazards workaround was standing in for a gap worth closing directly rather than routing around indefinitely. `rpsurv` is developed at github.com/ielbadisy/rpsurv.